

[← Go Back](#)

Hardware

Nano RP2040 Connect

Tutorials

[Nano RP2040 Connect User Manual](#)[Web Server AP Mode with Arduino Nano RP2040 Connect](#)[BLE Device to Device with Nano RP2040 Connect](#)[Nano RP2040 Connect Chromebook Setup](#)[Nano RP2040 Datalogger with MicroPython](#)[Using the IMU Machine Learning Core Features](#)[Accessing IMU Data on Nano RP2040 Connect](#)[Setting up Nano RP2040 Connect with Arduino Cloud](#)[Reading Microphone Data on Nano RP2040 Connect](#)[MicroPython and the IMU Machine Learning Core Features](#)[Getting Started With the Nano RP2040 Connect and OpenMV](#)[Control Built-in RGB LED over Wi-Fi with Nano RP2040 Connect](#)[Voice Commands With The Arduino Speech Recognition Engine](#)

Home / Hardware / Nano RP2040 Connect / **MicroPython and the IMU Machine Learning Core Features**



This tutorial refers to a product that has reached its end-of-life status.

MicroPython and the IMU Machine Learning Core Features

Learn how to use the Machine Learning Core (MLC) feature of the inertial module of the Nano RP2040 Connect.

Author · José Last Bagur · revision · 10/17/2022

ON THIS PAGE

Introduction

[Goals](#)[Hardware and Software Needed](#)[Inertial Measurement Unit](#)[Using the MLC for Vibration Monitoring](#) +[Vibration Monitoring Code](#) +[Testing Vibration Detection](#)[Troubleshoot](#)[Conclusion](#)

Introduction

In this tutorial, you will learn how to access the **Machine Learning Core** feature provided by the **LSM6DSOX**, the onboard Inertial Measurement Unit of the Arduino® Nano RP2040 Connect using **MicroPython** and the **OpenMV IDE**. The STMicroelectronics® Machine Learning Core combines several hardware and software tools that help designers implement gesture and activity recognition with Artificial Intelligence at the Edge in sensors through

[Help](#)

based on decision tree classifiers.

Note: Please refer to [this installation guide](#) if you need help setting up your environment to use your Nano RP2040 Connect board with MicroPython and the OpenMV IDE.



Goals

The goals of this tutorial are:

Detect motion activity such as vibration and head gestures using the Machine Learning Core feature of the Nano RP2040 Connect onboard inertial measurement unit.

Print motion activity in the Serial Terminal of the OpenMV IDE.

Hardware and Software Needed

[Arduino Nano RP2040 Connect board](#).

[OpenMV IDE](#).

Micro-USB cable (x1).

Inertial Measurement Unit

An Inertial Measurement Unit

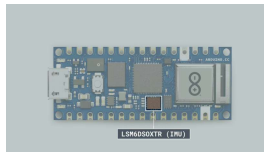
gravity and angular rate.
Typically, an IMU consists of
three devices:

Gyroscope: a device that
measures the system's
angular rate.

Accelerometer: a device
that measures the
system's specific
force/acceleration.

Magnetometer: an
optional device that
measures the surrounding
magnetic field of the
system.

IMU's are typically used to
capture **movement** data of a
device or a system. The onboard
IMU of the Nano RP2040
Connect, the LSM6DSOX from
STMicroelectronics®, has an
embedded hardware processing
engine dedicated to real-time
machine learning computing;
this means that some machine
learning algorithms were moved
from the application processor
to the LSM6DSOX directly.
STMicroelectronics named this
embedded processing engine of
the LSM6DSOX **Machine
Learning Core (MLC)**.



The LSM6DSOX
Inertial Measurement
Unit (IMU).

In the MLC, machine learning
processing is implemented

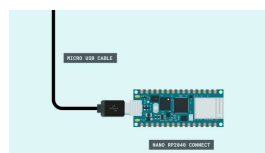
tool composed of a series of configurable nodes; each node is characterized by an "if-then-else" condition where an input signal (represented by statistical parameters calculated from the sensor data) is evaluated against a certain threshold. The decision tree results can be read from the application processor at any time and there is also the possibility to generate an interrupt for every change in the result in the decision tree.

Using the MLC for Vibration Monitoring

We will use the LSM6DSOX accelerometer from our Nano RP2040 Connect board, its MLC, and a pre-trained model to implement a simple **vibration monitoring** script. With this script, and the pre-trained model, **three** types of vibrations can be detected:

- 1 No vibration.
- 2 Low vibration.
- 3 High vibration.

Circuit



Plug in your Nano

board to your
computer.

Accelerometer Configuration

For the vibration monitoring application, the LSM6DSOX accelerometer is configured with **±4g full scale** and a **26 Hz output data rate**; any sensor orientation is allowed.

Machine Learning Core Configuration

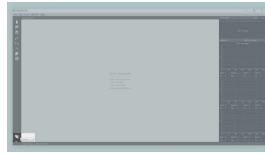
For the vibration monitoring application, just **one feature** has been used (peak-to-peak) and applied to the accelerometer norm squared input. The **MLC runs at 26 Hz**, computing features on windows of 16 samples (more or less, every 0.6 seconds). **One decision tree with just two nodes** has been configured to detect the different classes; an **interrupt** (pulsed and active high) is generated every time a new vibration type is detected.

Vibration Monitoring Code

We will now get to the programming part of this tutorial.

1. Setting Up

computer and open the OpenMV IDE; press the "Connect" button that is located in the lower left corner of the IDE.



"Connect" button in the OpenMV IDE.

The "Connect" button should change now. When the board is connected, information about it should be displayed in the lower status bar. **Make sure that the latest OpenMV firmware is uploaded to your board**, you can check the latest OpenMV firmware release [here](#).



Board information in the OpenMV IDE.

Now, **download the pre-trained model** for the vibration monitoring application and copy it to your Nano RP2040 Connect board storage. The pre-trained model for the vibration monitoring can be found [here](#) as well other pre-trained models for different examples and applications.

2. Creating the Script

First, let's **import some**

help you control your board:

```
1 import time
2 from lsm6dsox import L
3 from machine import Pin
```

Now, let's **enable interrupts**. At start, the interrupt flag is set to `FALSE` (this means that there are no pending interrupts at start):

```
1 INT_MODE = True
2 INT_FLAG = False
```

Now, let's **define the IMU interrupt handler**; the interrupt handler is a function that is going to be executed when an interrupt from the IMU is generated. This function, when executed, is going to update the value of `INT_FLAG` to `True`:

```
1 # Interrupt handler function
2 def imu_int_handler(pin):
3     # This function does
4     global INT_FLAG
5     INT_FLAG = True
```

Now, let's **configure the IMU interrupt** in our board. For configuring the interrupt, we need to specify the interrupt pin

the type of trigger of the interrupt (**rising edge**, this means a change from low to high):

```
1 # External interrupt c
2 if (INT_MODE == True):
3     int_pin = Pin(24)
4     int_pin.irq(handle
```

Now, let's create a **I2C object**, passing pins 13 and 12 for SCL and SDA:

```
1 # I2C object initializ
2 i2c = I2C(0, scl=Pin(1
```

Now, let's **configure the pre-trained model** for the vibration monitoring application. For configuring the pre-trained model in our sketch, we must define first the name of the file we copied into our Nano RP2040 Connect board storage before, the labels of the model and then initialize an IMU object with this file; notice that the selected data rate and scale matches the MLC data rate and scale described before [here](#):

```
1 # Pre-trained model de
2 UCF_FILE = "lsm6dsox_v
3 UCF_LABELS = {0: "no v
4 lsm = LSM6DSOX(i2c, gy
```


Now that we have the IMU ready, its time to start getting interrupts from it when a certain type of vibration is recognized. For this, we are going to keep looking for the interrupt flag `INT_FLAG` to change from `FALSE` to `TRUE`.

If the value of `INT_FLAG` is `TRUE`, the MLC output is read and translated into a human readable description of the detected vibration:

```
1 print("\n-----")
2 print("- Vibration Mo")
3 print("-----")
4 print("- MLC configur")
5
6 while (True):
7     if (INT_MODE):
8         if (INT_FLAG):
9             # Interru
10            INT_FLAG
11            print("-")
12        else:
13            buf = lsm.read
14            if (buf != No
15            print(UCF
```

Complete Script

If you choose to skip the code building section, the complete script can be found below:



```

1  # LSM6DSOX IMU MLC
2  # Application: Vibr
3  # Original example
4
5  import time
6  from lsm6dsiox impor
7  from machine import
8
9  INT_MODE = True
10 INT_FLAG = False
11
12 # Define the interr
13 def imu_int_handler
14     global INT_FLAG
15     INT_FLAG = True
16
17 # Configure the ext
18 if (INT_MODE == Tru
19     int_pin = Pin(2
20     int_pin.irq(han
21
22 # Initialize an I2C
23 i2c = I2C(0, scl=Pi
24
25 # Pre-trained model
26 # NOTE: Selected da
27 UCF_FILE = "lsm6dso
28 UCF_LABELS = {0: "no

```

Now you are ready to upload and run the script to your board. Press the green "Start" button that is located in the lower left corner of the IDE.

Testing Vibration Detection

After successfully uploading the code to the board, open OpenMV IDE's Serial Terminal. If the board is not moving at all, e.g. standing still at a table, the Serial Terminal will print

No vibration after a few

continuously, the Serial Terminal will print `Low vibration` or `High vibration`, depending on how your board is moving.



The Serial Terminal of the OpenMV IDE.

Troubleshoot

If the code is not working, there are some common issues you can troubleshoot:

- Your board doesn't have uploaded the latest release of the OpenMV firmware.

- Your board doesn't have uploaded the pre-trained model into its storage.

- An incorrect pin number was assigned to the IMU interrupt. Make sure that pin 24 is assigned to it.

- You have a faulty Micro-USB cable; try with a different cable.

Conclusion

In this tutorial, you have learned how to use the Nano RP2040 Connect IMU's MLC with a pre-trained model to detect various activity types. With these embedded machine learning capabilities, certain types of

by the onboard IMU of the Nano RP2040 Connect. If you want more learn more about the MLC from STMicroelectronics, check out this [repository](#) from STMicroelectronics on GitHub that provides detailed information, examples and configurations of the MLC.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

